



Grafické uživatelské rozhraní v režimu okamžitého vykreslování

Bc. Radek Mocek

Ing. Jiří Jeníček, Ph.D.

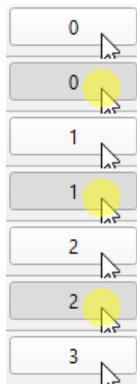
Cíle diplomové práce

1. Proved'te rešerši GUI v režimu okamžitého vykreslování.
2. Navrhněte ukázkovou aplikaci.
3. Ve vybraných knihovnách implementujte a otestujte aplikaci.
4. Zhodnoťte výsledky.

RMGUI – Retained Mode GUI

- „Režim soběstačného vykreslování“
- V inicializační fázi se definují jednotlivé ovládací prvky a na ně se navěšují callback metody
- Hlavní smyčka pak řeší vykreslování prvků a reakce na události, před programátorem je skryta
- Knihovny *Qt*, *GTK*, *wxWidgets*, *WPF*, ...

```
fn init():  
    m_value = 0  
    m_button = new GUI::Btn(str(m_value))  
    layout.add_widget(m_button)  
    connect(m_button, GUI::Btn::click, button_click)  
  
fn button_click():  
    m_value += 1  
    m_button.set_text(str(m_value))
```



IMGUI – Immediate mode GUI

- „Režim okamžitého vykreslování“
- Žádná inicializační fáze ani callback metody, pouze hlavní smyčka
- Knihovny *Dear ImGui*, *egui*, *Nuklear*, ...

```
fn update():  
    static value = 0  
    if GUI::btn(str(value)):  
        value += 1
```

- Definice zmiňuje pouze tento rozdíl z pohledu programátora koncové aplikace, implementace knihoven mohou být různé.
- V praxi ale:
 - Využití grafického API pro vykreslování ovládacích prvků (*OpenGL*, *DirectX*, ...)
 - Časté použití pro přidání ovládacích prvků do existující interaktivní aplikace
 - Spíše GUI pro ladění vývojářem, než user-facing GUI
 - Chaos ohledně toho, co je a není IMGUI

Motivace

- Jak jsou ImGui knihovny použitelné pro psaní „běžných“ user-facing aplikací?
- Jaký je dopad na výkon oproti RMGUI knihovnám?
- ImGui knihovny jsou v aktivním vývoji
- Vnést do chaosu trochu řádu

Návrh ukázkové aplikace

- Program, který vykreslí diagram na základě dodaného strukturovaného popisu
- $\implies$ víceřádkové textové pole a vykreslování 2D primitiv
- $\implies$ praktická funkce, ne jen „galerie ovládacích prvků“
- Dále se snažit prozkoumat různé vlastnosti vybraných knihoven a přidat uživatelem očekávané funkce (lišta nabídek, dialog pro uložení, nastavení vzhledu)
- Implementovat v knihovnách *Dear ImGui*, *egui* a *Qt*

Ukázková aplikace

File View Debug Help

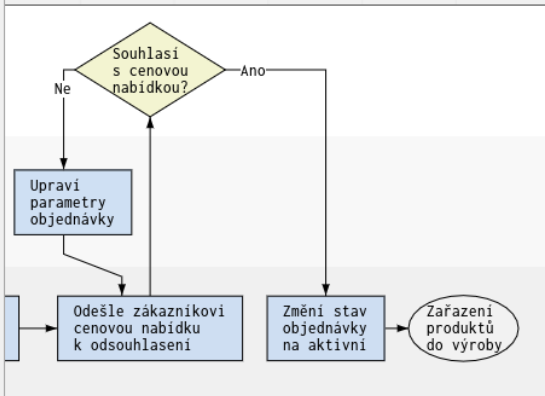
Font size: 20 - + Cursor pos: 22,38 Node color:  Type: ☐ Ellipse ▼ ID: s7

```
[node.bg_bg]
z = 2
size = [1602, 420]

[node.bg_z]
z = 3
value = "Zákazník"
size = ["bg_width", "bg_height"]
label_pos = "top-left"
xy = ["bg_bg", "top-left", 1, 1]
color = "#ffffff"
color_border = "#ffffff00"
label_shift = ["bg_label_shift", "abcd"]

[node.bg_p]
z = 3
value = "Pracovník"
size = ["bg_width", "bg_height"]
label_pos = "top-left"
pivot = "top"
```

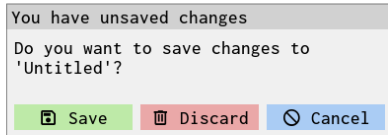
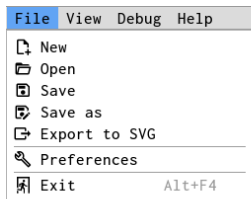
Variable 'abcd' does not exist



```
graph TD
    Start([Start]) --> SendOffer[Odešle zákazníkovi cenovou nabídku k odsouhlasení]
    SendOffer --> Decision{Souhlasí s cenovou nabídkou?}
    Decision -- Ne --> EditParams[Upraví parametry objednávky]
    EditParams --> SendOffer
    Decision -- Ano --> ChangeStatus[Změni stav objednávky na aktivní]
    ChangeStatus --> AssignProd([Zařazení produktů do výroby])
```

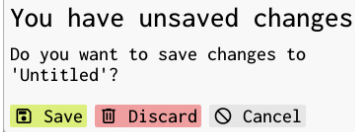
The flowchart illustrates a process for handling a customer's price offer. It begins with a start node leading to a rectangular process box labeled "Odešle zákazníkovi cenovou nabídku k odsouhlasení" (Send customer price offer for approval). This leads to a yellow diamond decision box labeled "Souhlasí s cenovou nabídkou?" (Agrees with price offer?). If the answer is "Ne" (No), the flow goes to a rectangular process box "Upraví parametry objednávky" (Adjust order parameters), which then loops back to the "Odešle..." box. If the answer is "Ano" (Yes), the flow goes to a rectangular process box "Změni stav objednávky na aktivní" (Change order status to active), which then leads to an oval terminal node "Zařazení produktů do výroby" (Assign products to production). The diagram is displayed in a software interface with a toolbar at the bottom containing icons for rectangle, ellipse, diamond, and text, along with a zoom level of 0.78.

Ukázková aplikace



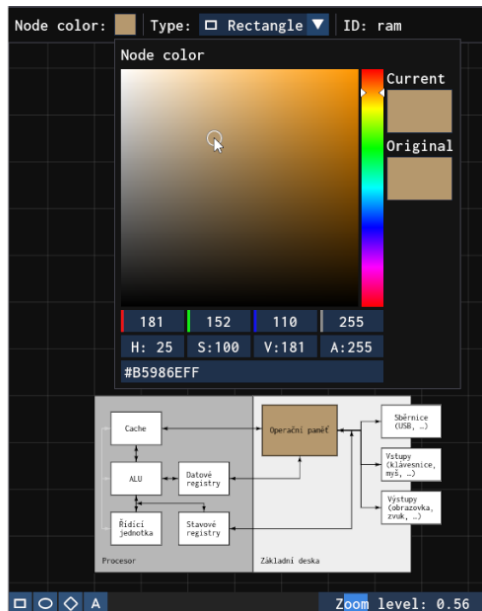
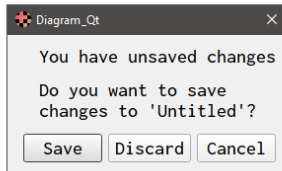
File View Debug Help

- New
- Open
- Save
- Save as
- Export to SVG
- Preferences
- Exit



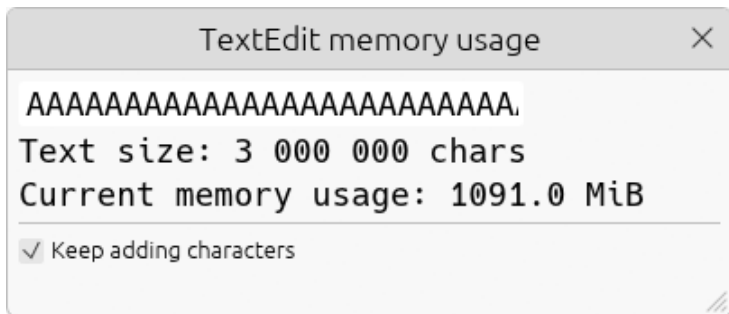
File View Debug Help

- New
- Open
- Save
- Save as
- Export to SVG
- Preferences
- Exit



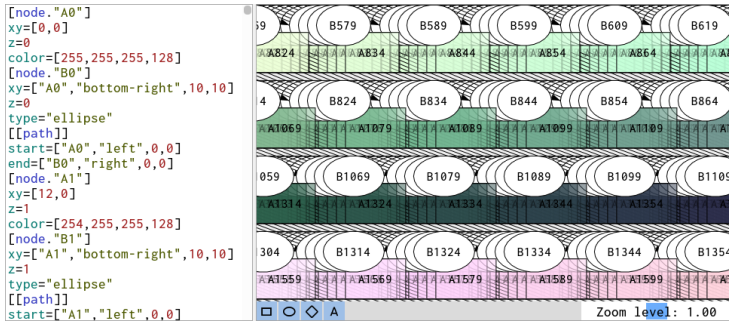
IMGUI a textová pole

- Textové pole je komplexní ovládací prvek, který má napříč IMGUI knihovnami omezené funkce
- (Pohyb kurzorem, vyjmout, kopírovat, vložit, zpět, vpřed, více řádků, ...)
- *Dear ImGui* používá na pozadí *stb_textedit*, v plánu je kompletní přepsání
- *egui* má problém s využitím operační paměti:

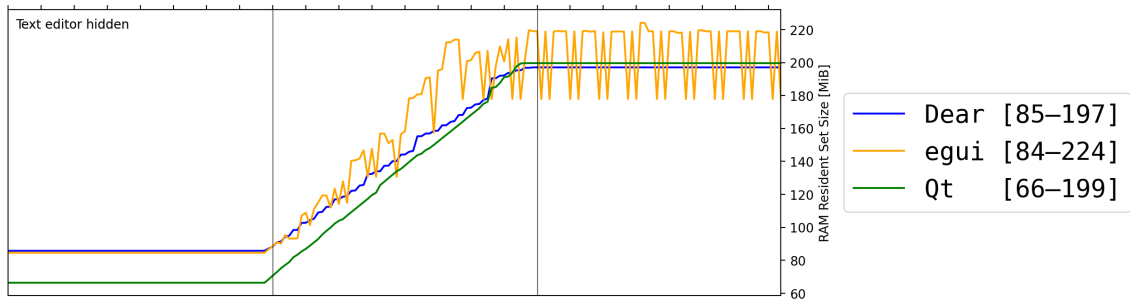
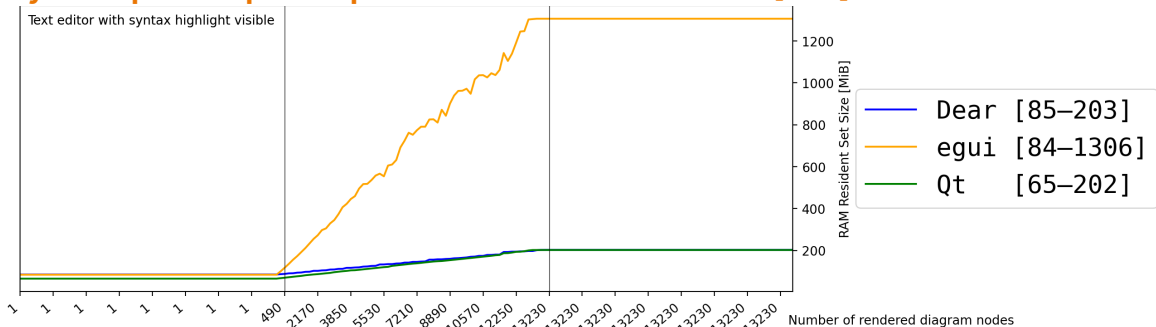


Benchmark vykreslování uzlů

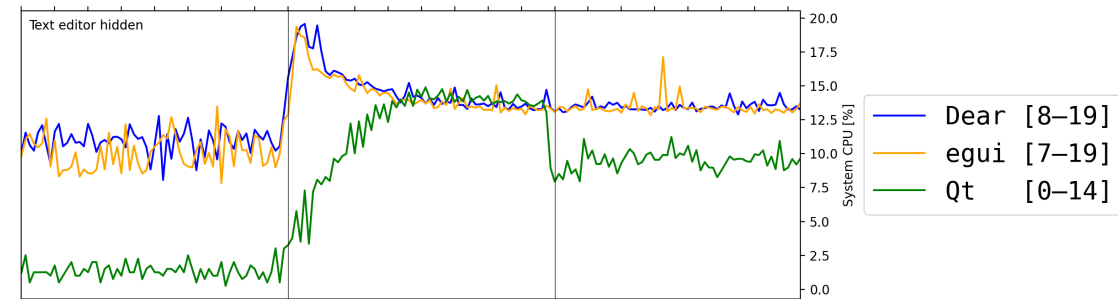
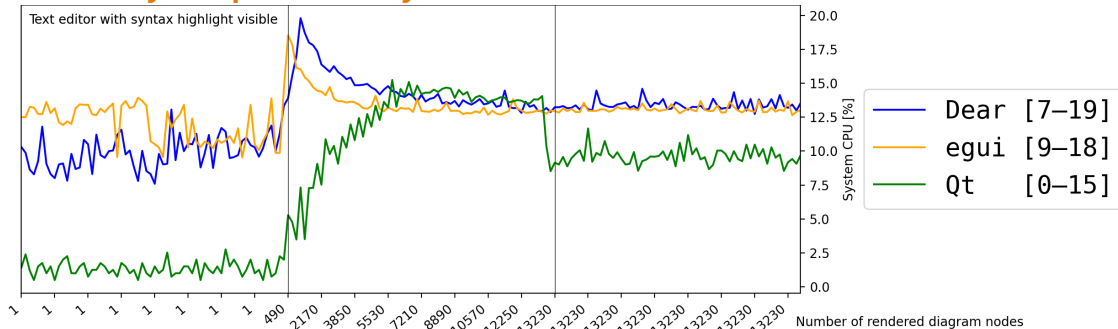
- Každých 300 ms mírně posune scénou a změni úroveň přiblížení
- Každých 900 ms zaznamená spotřebu RAM a CPU
- Benchmark má tři fáze:
 - Fáze 1: Jeden uzel ve scéně
 - Fáze 2: Postupné přidávání uzlů
 - Fáze 3: 13 230 uzlů ve scéně
- Následují výsledky měření pro Laptop:
Intel Core i7 6700HQ (2015), 16 GB RAM, Linux Mint 22



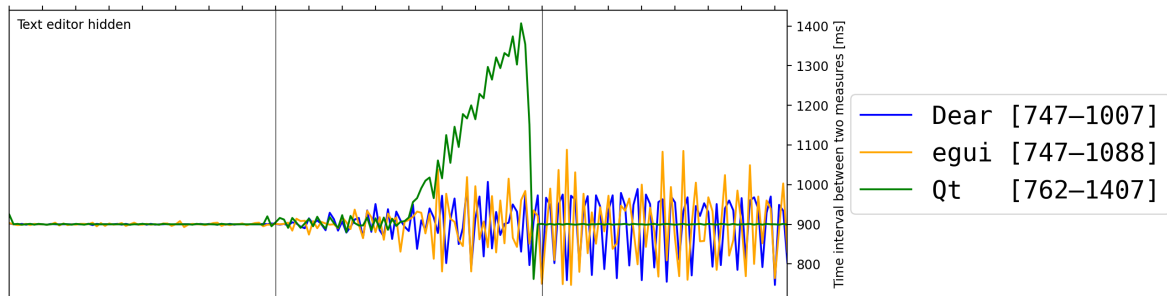
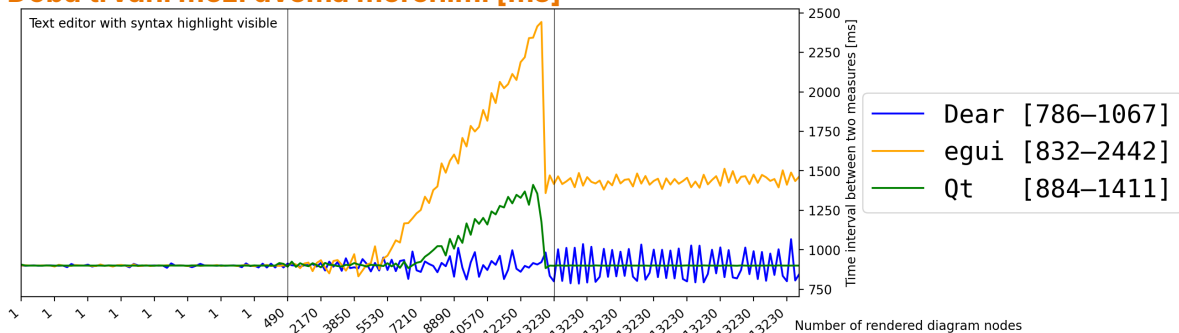
Využití operační paměti procesem – Resident Set Size [MiB]



Celkové využití procesoru systémem [%]

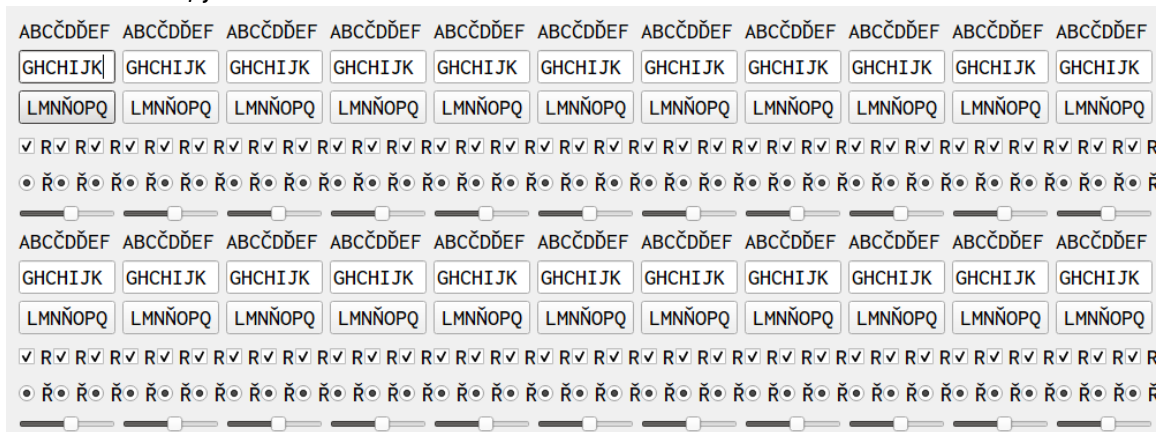


Doba trvání mezi dvěma měřeními [ms]



Benchmark ovládacích prvků

- Okno obsahující velké množství základních ovládacích prvků
- Měří dobu potřebnou pro zobrazení a uzavření tohoto okna
- Jedna dávka obsahuje 172 ovládacích prvků
- S každou iterací se zdvojnásobí počet dávek
- Test končí, jakmile se čeká déle než 15 vteřin



Benchmark ovládacích prvků

Pčt. widgetů	Doba trvání [ms]			Využití RAM [MiB]		
	Dear	egui	Qt	Dear	egui	Qt
172	14	16	8	86	85	66
344	16	16	11	86	86	66
688	13	16	18	87	88	67
1 376	16	18	37	89	86	69
2 752	17	18	75	89	87	73
5 504	18	21	157	91	92	81
11 008	22	23	378	91	95	97
22 016	19	37	939	93	100	128
44 032	21	62	2 656	94	112	191
88 064	26	107	9 026	94	131	318

Benchmark ovládacích prvků

Pčt. widgetů	Doba trvání [ms]			Využití RAM [MiB]		
	Dear	egui	Qt	Dear	egui	Qt
176 128	31	197	—	96	171	—
352 256	33	408	—	96	253	—
704 512	58	848	—	98	420	—
1 409 024	108	1 803	—	98	749	—
2 818 048	211	3 930	—	100	1 393	—
5 636 096	414	8 569	—	100	2 662	—
11 272 192	820	—	—	102	—	—
22 544 384	1 635	—	—	103	—	—
45 088 768	3 258	—	—	104	—	—
90 177 536	6 535	—	—	104	—	—
180 355 072	12 996	—	—	104	—	—

Závěr

V rámci práce jsem provedl:

- Rešerši dvou přístupů ke psaní GUI aplikací: RMGUI a IMGUI
- Rešerši existujících IMGUI knihoven
- Navrhl a implementoval ukázkovou aplikaci: editor diagramů v knihovnách *Dear ImGui*, *egui* a *Qt*
- Implementoval dva benchmarky a porovnal v nich výsledné aplikace
- Shrnl své poznatky o zkoumaných přístupech a knihovnách, ve zkratce:
 - Přístup RMGUI je o programování callbacků, které mohou nečekaně ovlivnit chování aplikace po přidání nových funkcí.
 - Přístup IMGUI je o programování v nekonečné smyčce, kdy je potřeba přemýšlet nad pořadím provedených operací.
 - V RMGUI synchronizujeme stav aplikace se stavem knihovny, v IMGUI máme veškerý stav na straně aplikačního kódu.
 - Přístup IMGUI vede na kratší kód a rychlejší vývoj.
 - Knihovny s přístupem IMGUI oproti zavedeným RMGUI knihovnám v určitých aspektech zaostávají.



Děkuji za pozornost